

Question 2.i:

- We analyze the effect of TCP window size on throughput and retransmissions between nodes n7 (client) and n11 (server) using iperf3 with 40 second duration. Results are shown in (Table 1) and (Figure 1).

Window Size (KB)	Avg Throughput (Kbits/sec)	Total avg Retransmissions
1	139	0
2	362	0
3	718	0
4	722	0
5	720	0
6	96.4	278
12	449	350
16	432	310
24	173	329
32	147	326

Table 1: Average Throughput and Average Retransmissions vs TCP Window Size (n7 to n11)

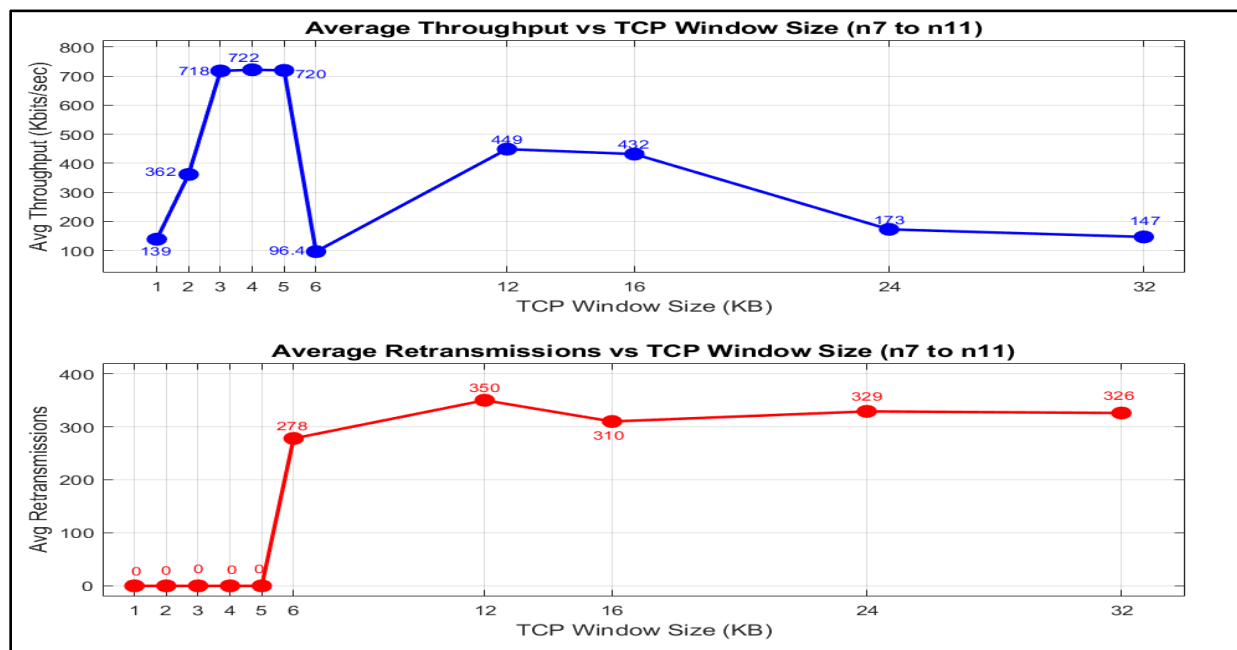


Figure 1 Average Throughput and Retransmissions vs TCP Window Size

- Comment:
 - For window sizes (1KB: 2KB): the window fits within the network buffer capacity.
 - At 6 KB: buffer overflow begins throughput collapses to 96.4 Kbits/sec and retransmissions spike to 278.
 - For larger windows (12–32 KB): the zigzag pattern results from TCP's AIMD (Additive Increase, Multiplicative Decrease) congestion control cycle, where oversized windows overflow router buffers causing packet drops, TCP then multiplicatively decreases its rate, then additively increases again causing congestion to recur, and this cycle repeats creating oscillation with an overall declining trend.

Question 2.ii:

- We choose (Frame 219602) sent from node n7 to node n11 that has been captured by Wireshark, the length of the frame is **1516 bytes**, broken down as shown in (Table 2):

Layer	Protocol	Header Size (bytes)
Transport	TCP	32 (20+ 12 options)
Network	IPv4	20
Data Link	Linux SLL (cooked capture)	16
Data Payload		1448
TOTAL		1516

Table 2: Protocol Layer Header Breakdown

- TCP options consist of two NOP (No-Operation) and Timestamps, as shown in (Table 3)

TCP options used	Size(bytes)
NOP (No-Operation) ×2	2
Timestamps	10

Table 3: TCP Options Identified

- This is also screenshot from Wireshark showing TCP header and TCP options.

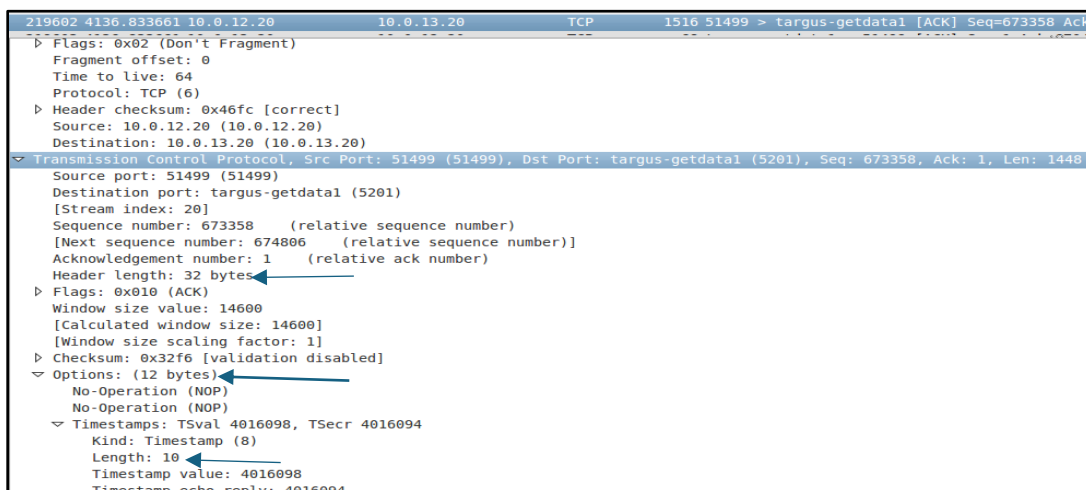


Figure 2 Wireshark TCP Header and TCP Options (n7 to n11)

- This is also screenshot from Wireshark showing IPv4 header length.

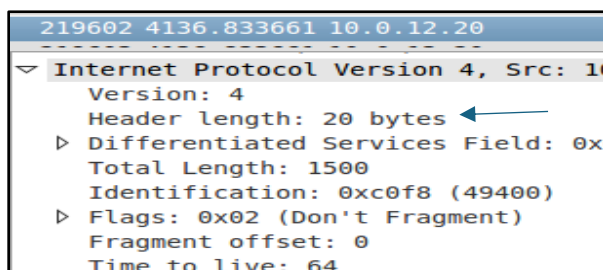


Figure 3 Wireshark IPv4 Header (n7 to n11)

- Data payload screenshot:

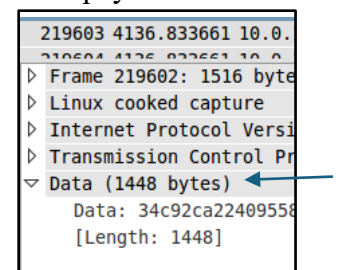


Figure 4 Data Payload Screenshot (n7 to n11)

Question 2.iii:

- We choose (Frame 219568) sent from node n11 to node n7 that has been captured by Wireshark, the length of the frame is **68 bytes**, broken down as shown in (Table 4):

Layer	Protocol	Header Size (bytes)
Transport	TCP	32 (20+ 12 options)
Network	IPv4	20
Data Link	Linux SLL (cooked capture)	16
Data Payload		0 (pure ACK)
TOTAL		68

Table 4: Protocol Layer Header Breakdown

- TCP options consist of two NOP (No-Operation) and Timestamps, as shown in (Table 5)

TCP options used	Size(bytes)
NOP (No-Operation) ×2	2
Timestamps	10

Table 5: TCP Options Identified

- screenshot from Wireshark showing TCP header and TCP options.

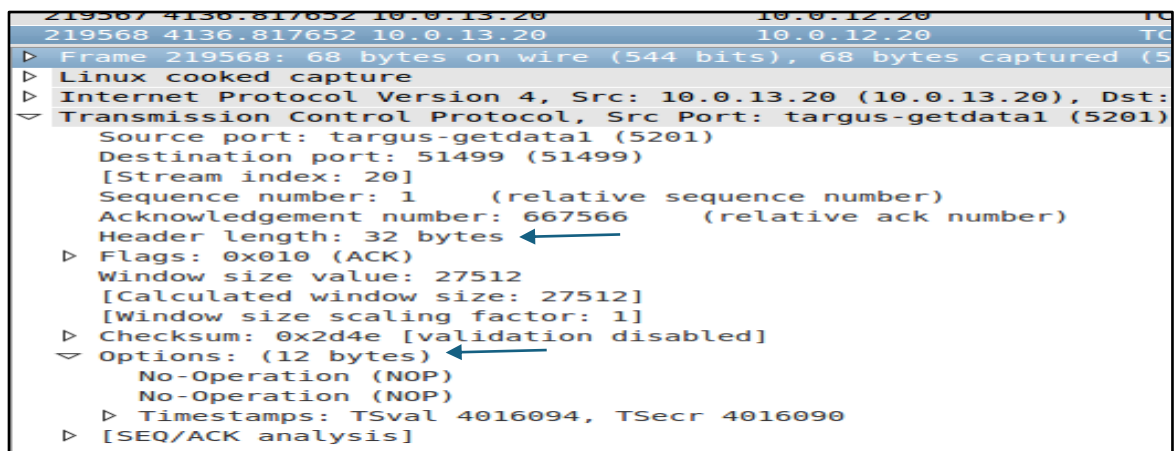


Figure 5 Wireshark TCP Header and TCP Options (n11 to n7)

- screenshot from Wireshark showing IPv4 header length.

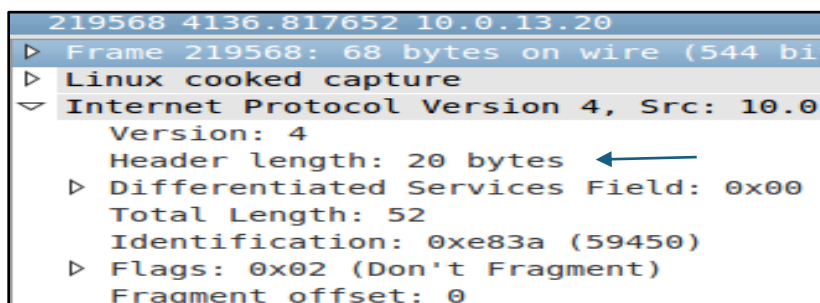


Figure 6 Wireshark IPv4 Header (n11 to n7)

Question 3.i:

- By comparing the result of throughput, we will find that the average throughput from n7 to n8 is less than that from n7 to n11, as shown in (Table 6 & Figure 7):

Connection	Window Size (KB)	Avg Throughput (Kbits/sec)	Retransmissions
From n7 to n11 (Short Path)	4	722	0
From n7 to n8 (Long Path)	4	482	0

Table 6: Throughput Comparison (n7→n11 vs n7→n8)

<pre> root@n7:/tmp/pycore.47733/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k Connecting to host 10.0.13.20, port 5201 [4] local 10.0.12.20 port 51487 connected to 10.0.13.20 port 5201 [ID] Interval Transfer Bandwidth Retr Cwnd [4] 0.00-10.00 sec 880 KBytes 720 Kbits/sec 0 14.1 KBytes [4] 10.00-20.00 sec 880 KBytes 720 Kbits/sec 0 14.1 KBytes [4] 20.00-30.00 sec 882 KBytes 723 Kbits/sec 0 14.1 KBytes [4] 30.00-40.00 sec 882 KBytes 723 Kbits/sec 0 14.1 KBytes ----- [ID] Interval Transfer Bandwidth Retr [4] 0.00-40.00 sec 3.44 MBytes 722 Kbits/sec 0 [4] 0.00-40.00 sec 3.44 MBytes 721 Kbits/sec iperf Done. root@n7:/tmp/pycore.47733/n7.conf# </pre>	<pre> root@n7:/tmp/pycore.47733/n7.conf# iperf3 -c 10.0.11.20 -t 40 -i 10 -w 4k Connecting to host 10.0.11.20, port 5201 [4] local 10.0.12.20 port 59848 connected to 10.0.11.20 port 5201 [ID] Interval Transfer Bandwidth Retr Cwnd [4] 0.00-10.00 sec 591 KBytes 484 Kbits/sec 0 14.1 KBytes [4] 10.00-20.00 sec 588 KBytes 482 Kbits/sec 0 14.1 KBytes [4] 20.00-30.00 sec 588 KBytes 482 Kbits/sec 0 14.1 KBytes [4] 30.00-40.00 sec 588 KBytes 482 Kbits/sec 0 14.1 KBytes ----- [ID] Interval Transfer Bandwidth Retr [4] 0.00-40.00 sec 2.30 MBytes 482 Kbits/sec 0 [4] 0.00-40.00 sec 2.30 MBytes 482 Kbits/sec iperf Done. root@n7:/tmp/pycore.47733/n7.conf# </pre>
---	---

Figure 7 Throughput Comparison (n7→n11 vs n7→n8)

- The reason for dropping throughput when connecting to n8 is because the Path is **longer** (more hops), resulting in a higher **Round Trip Time (RTT)**. Since TCP throughput is governed by: $\text{Throughput} = \frac{\text{Window Size}}{\text{RTT}}$
 - A larger RTT with the same window size (4KB) directly reduces the throughput, explaining why n8 achieves lower performance than n11 despite identical link capacities.
 - With window size fixed at 4KB, the sender can only have 4KB "in-flight" before waiting for an ACK. The longer path to n8 increases propagation delay, causing the sender to spend more time idle and reducing link utilization, this is known as the **Bandwidth-Delay Product (BDP)**.

Question 4:

- After running iperf3 client from n7 to n11 with window size 4K, we got these results for each case, as shown in (Table 7)

Case	Capacity (Mbps)	Loss	Avg Throughput (Kbits/sec)	Retransmissions
a	10	0%	723	0
b	3	0%	719	0
c	10	5%	271	85
d	100	10%	102	57
e	10	1% (n4→n5), 0% (n5→n4)	677	10
f	10	0% (n4→n5), 1% (n5→n4)	623	21

Table 7: Throughput Results for All Cases (n7 → n11, Window Size 4KB)

Question 4.i:

- **Case (b) is better than case (c)**, because case (b) achieved (719 Kbits/sec), case (a) achieved (723 Kbits/sec) and case (c) achieved only (271 Kbits/sec), as shown in (Table-7).
 - Although case (b) has lower capacity (3 Mbps) than case c (10 Mbps), it achieves much higher throughput because it has **zero loss**.
 - Case c's 5% loss causes frequent retransmissions (85 retransmissions), triggering TCP's congestion control to repeatedly reduce the transmission rate. This demonstrates that **reliability is more important than raw capacity** for TCP performance.
-

Question 4.ii:

- Comparing cases b, c, and d as shown in (Table 7), we found that:
 - Case **b** (3 Mbps, 0% loss): 719 Kbits/sec, 0 retransmissions.
 - Case **c** (10 Mbps, 5% loss): 271 Kbits/sec, 85 retransmissions.
 - Case **d** (100 Mbps, 10% loss): 102 Kbits/sec, 57 retransmissions.
 - **Case (b) is the best** despite having the lowest capacity (3 Mbps) because its **zero loss** allows TCP to maintain a steady flow without any congestion control penalties.
 - **Case(d) performs the worst** despite having the highest capacity (100 Mbps) because its **10% loss rate** is catastrophic for TCP, every lost packet triggers AIMD to drastically reduce the congestion window, making the sender unable to utilize even a fraction of the available capacity.
-

Question 4.iii:

- Comparing cases e and f as shown in (Table 7), we found that:
 - Case **e** (1% loss $n4 \rightarrow n5$, 0% loss $n5 \rightarrow n4$): 677 Kbits/sec, 10 retransmissions.
 - Case **f** (0% loss $n4 \rightarrow n5$, 1% loss $n5 \rightarrow n4$): 623 Kbits/sec, 21 retransmissions.
- **Case (e) is better than case (f)** despite both having the same capacity and same total loss rate (1%), because in case **e** the loss occurs on the **data path** ($n4 \rightarrow n5$), meaning only data packets are dropped and retransmitted directly.
- In case **f**, the loss occurs on the **ACK path** ($n5 \rightarrow n4$), meaning ACKs are lost, this causes the sender to **timeout and retransmit data that was already successfully received** by $n11$, wasting bandwidth and generating more retransmissions (21 vs 10).

Question 5.i: Default OSPF Path and Effect of Cost Change on n5

- We ran a long iperf3 session from n7 to n11 (window size 4K, duration 500s), then used traceroute to observe the default OSPF path before any cost changes, as shown in (Figure 8).

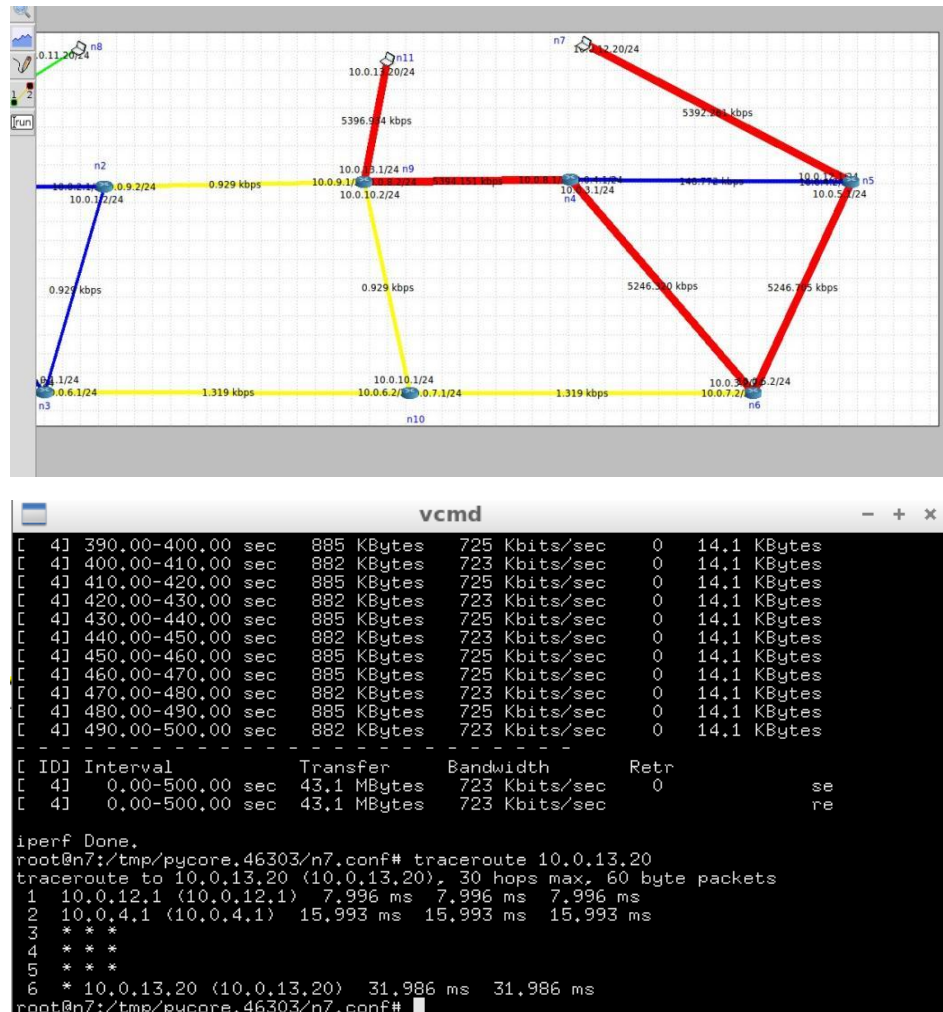


Figure 8: Traceroute from n7 to n11 — Default Path (before cost change)

- The default path from n7 to n11 goes through: n7 → 10.0.12.1 → 10.0.4.1 → 10.0.13.20 (n11), with RTT ≈ 31.986 ms.
- show ip ospf interface eth1** on n5 to check the current cost (Cost: 10), then changed it to 40, as shown in (Figure 9).

```

vcmd
Hello, this is Quagga (version 0.99.20.1).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

n5# show ip ospf interface eth1
eth1 is up
  ifindex 117, MTU 1500 bytes, BW 0 Kbit <UP,BROADCAST,RUNNING,MULTICAST
  Internet Address 10.0.4.2/24, Area 0.0.0.0
  MTU mismatch detection:enabled
  Router ID 10.0.5.1, Network Type BROADCAST, Cost: 10
  Transmit Delay is 1 sec, State DR, Priority 1
  Designated Router (ID) 10.0.5.1, Interface Address 10.0.4.2
  Backup Designated Router (ID) 10.0.3.1, Interface Address 10.0.4.1
  Saved Network-LSA sequence number 0x80000002
  Multicast group memberships: OSPFA11Routers OSPFDesignatedRouters
  Timer intervals configured, Hello 10s, Dead 40s, Wait 40s, Retransmit
    Hello due in 7.023s
  Neighbor Count is 1, Adjacent neighbor count is 1
n5# configure terminal
n5(config)# interface eth1
n5(config-if)# ip ospf cost 40
n5(config-if)# exit
n5(config)# exit
n5#

```

Figure 9: show ip ospf interface eth1 on n5 and setting cost to 40

- After changing the cost on n5 eth1 to 40, OSPF recalculated the shortest path. The new traceroute shows the path changed, with RTT increasing to ≈ 92 ms, confirming OSPF rerouted traffic, as shown in (Figure 10).

```

vcmd
[ 4] 450.00-460.00 sec 885 KBytes 725 Kbits/sec 0 14.1 KBytes
[ 4] 460.00-470.00 sec 885 KBytes 725 Kbits/sec 0 14.1 KBytes
[ 4] 470.00-480.00 sec 882 KBytes 723 Kbits/sec 0 14.1 KBytes
[ 4] 480.00-490.00 sec 885 KBytes 725 Kbits/sec 0 14.1 KBytes
[ 4] 490.00-500.00 sec 882 KBytes 723 Kbits/sec 0 14.1 KBytes
-----
[ ID] Interval          Transfer      Bandwidth    Retr
[ 4] 0.00-500.00 sec 43.1 MBytes 723 Kbits/sec 0
[ 4] 0.00-500.00 sec 43.1 MBytes 723 Kbits/sec 0

iperf Done.
root@n7:/tmp/pycore.46303/n7.conf# traceroute 10.0.13.20
traceroute to 10.0.13.20 (10.0.13.20), 30 hops max, 60 byte packets
 1 10.0.12.1 (10.0.12.1) 7.996 ms 7.996 ms 7.996 ms
 2 10.0.4.1 (10.0.4.1) 15.993 ms 15.993 ms 15.993 ms
 3 * * *
 4 * * *
 5 * * *
 6 * 10.0.13.20 (10.0.13.20) 31.986 ms 31.986 ms
root@n7:/tmp/pycore.46303/n7.conf# traceroute 10.0.13.20
traceroute to 10.0.13.20 (10.0.13.20), 30 hops max, 60 byte packets
 1 10.0.12.1 (10.0.12.1) 8.001 ms 8.001 ms 8.001 ms
 2 * * *
 3 * * *
 4 10.0.13.20 (10.0.13.20) 68.005 ms 68.005 ms *
root@n7:/tmp/pycore.46303/n7.conf#

```

Figure 10: Traceroute from n7 to n11 — Before (31.986 ms) and After (92.005 ms) cost change on n5

- **Comment:** When the OSPF cost on n5 eth1 was increased from 10 to 40, the OSPF Dijkstra algorithm found an alternative lower-cost path, redirecting traffic through different intermediate routers. This increased the RTT from ~ 32 ms to ~ 92 ms, demonstrating that OSPF dynamically adapts routing based on link costs rather than raw bandwidth alone.

Question 5.ii: Two Simultaneous iperf3 Connections

- We restored the cost on n5 back to 10, then started two simultaneous iperf3 connections: n7 → n11 and n11 → n7, as shown in (Figure 11).

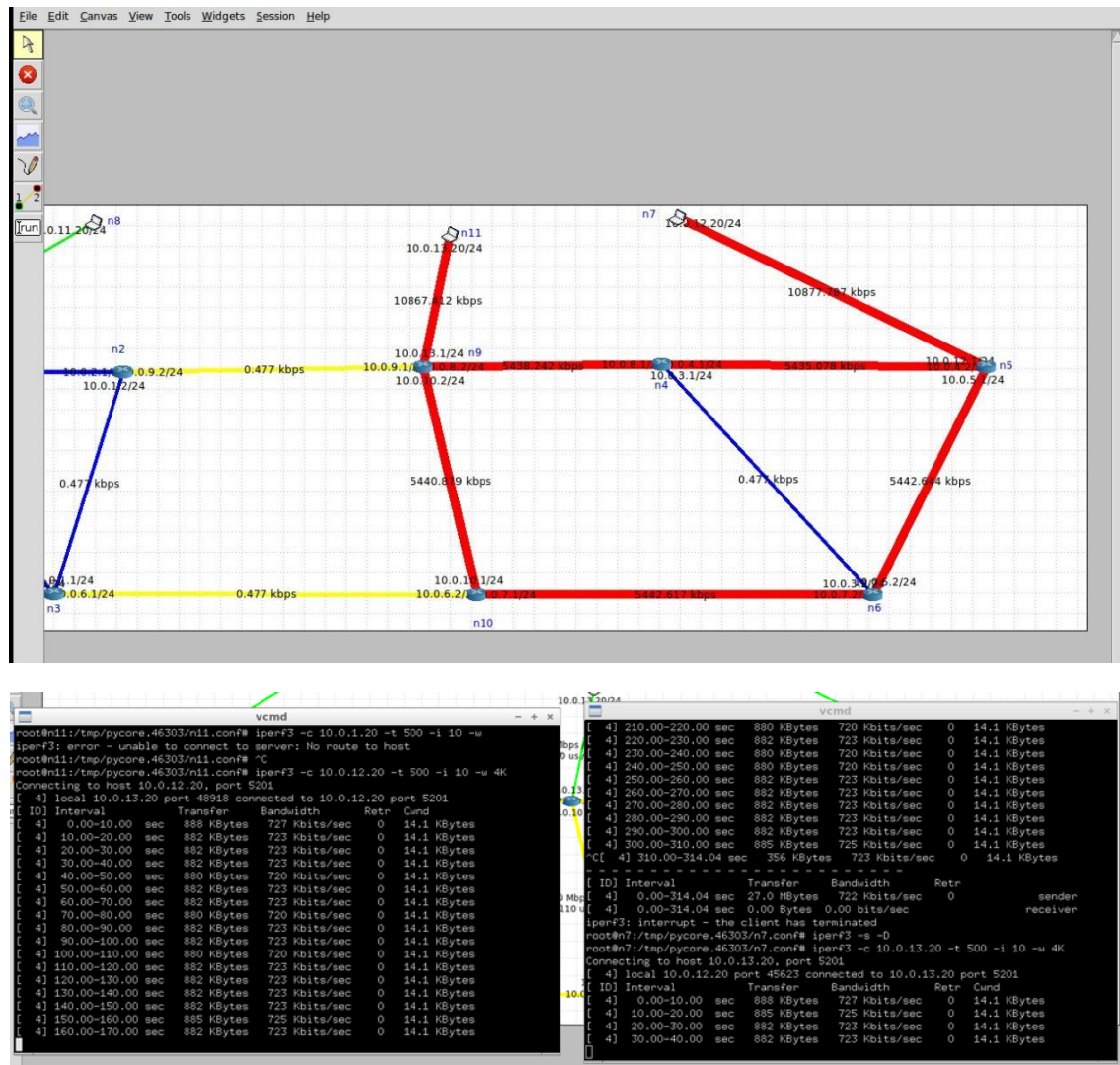


Figure 11: Two Simultaneous iperf3 Connections (n7↔n11), Avg. Throughput ≈ 722 Kbits/sec each

- Both connections achieved approximately 722 Kbits/sec with 0 retransmissions, sharing the available bandwidth symmetrically.
- We then increased the OSPF cost on n4 eth1 to 40. The traceroute from n7 showed the path changed (intermediate hops became unreachable via ICMP), and RTT increased to 92 ms, as shown in (Figure 12).

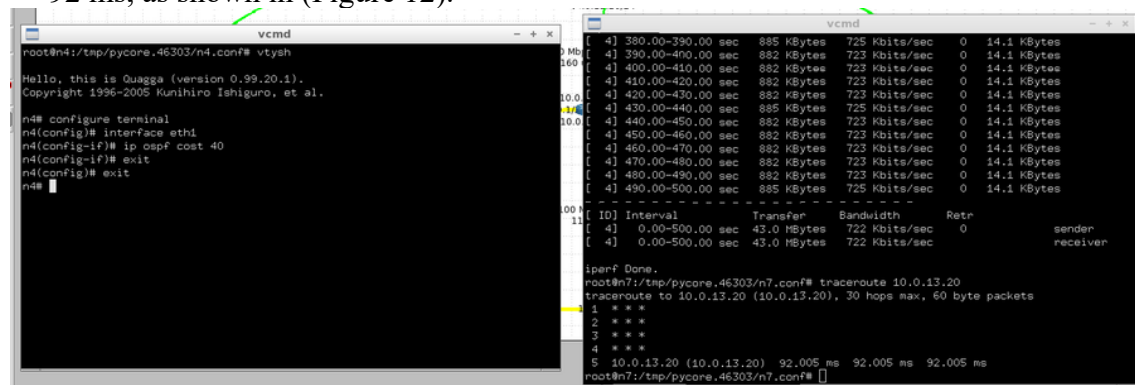


Figure 12: n4 OSPF cost set to 40 (left) and updated traceroute from n7 showing path change (right)

- **Comment:** Increasing the OSPF cost on n4 forced OSPF to reroute one or both directions through a different path. The RTT increased because the new path is longer. The throughput remained stable (~722 Kbits/sec) because the alternative path still had sufficient capacity, demonstrating OSPF's ability to perform automatic path failover without interrupting active sessions.

Question 6: OSPF Database Updates

Question 6.i: Wireshark Capture of OSPF Traffic

- We opened Wireshark on node n2 and applied the filter (ospf and not ospf.msg.hello) to capture only non-Hello OSPF packets. Initially, no packets were displayed, confirming the network was stable with only periodic Hello packets exchanged, as shown in (Figure 13).

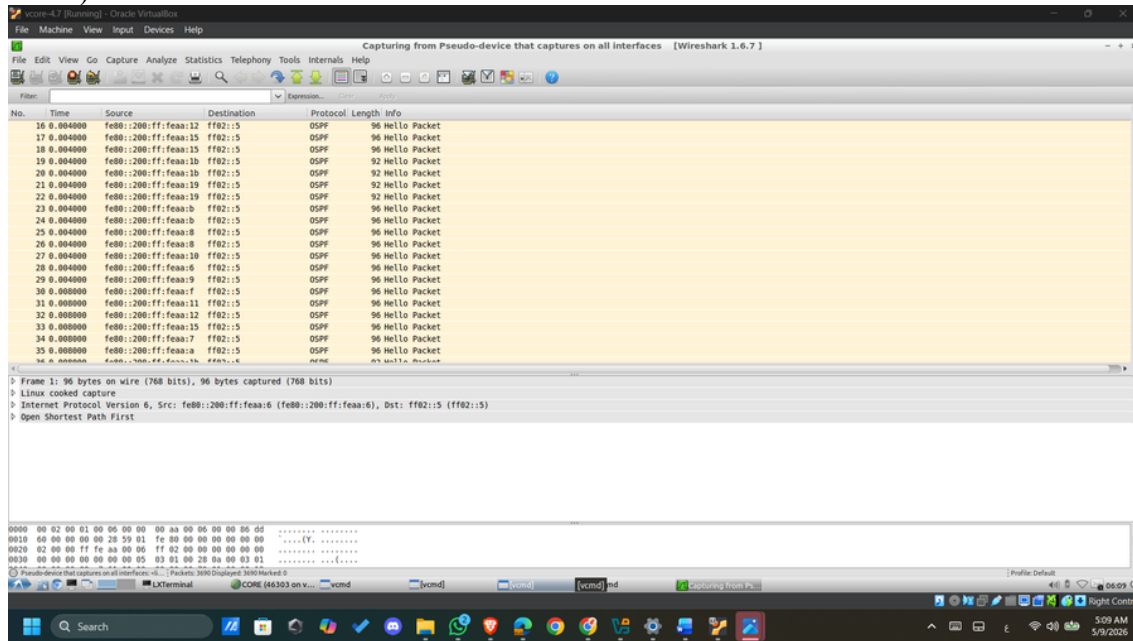


Figure 13: Wireshark on n2 — Only Hello Packets Before Any Change (stable state)

Question 6.ii: OSPF Database Before Changes

- We ran show ip ospf database and show ip ospf route on n2 to record the initial OSPF state, as shown in (Figures 14 and 15)

```
Hello, this is Quagga (version 0.99.20.1).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

n2# show ip ospf database

    OSPF Router with ID (10.0.1.2)

    Router Link States (Area 0.0.0.0)

Link ID      ADV Router   Age  Seq#       CkSum  Link count
10.0.0.1     10.0.0.1     771  0x8000000d 0xeccc 3
10.0.0.2     10.0.0.2     771  0x8000000d 0x940b 3
10.0.1.2     10.0.1.2     780  0x8000000d 0x870d 3
10.0.3.1     10.0.3.1     412  0x80000010 0xb3d5 3
10.0.3.2     10.0.3.2     823  0x8000000e 0xa5e2 3
10.0.5.1     10.0.5.1     663  0x8000000c 0x8c09 3
10.0.6.2     10.0.6.2     782  0x8000000c 0xc6ae 3
10.0.8.2     10.0.8.2     791  0x8000000e 0x1e17 4

    Net Link States (Area 0.0.0.0)

Link ID      ADV Router   Age  Seq#       CkSum
10.0.0.2     10.0.0.2     1771 0x80000003 0x5bcd
10.0.1.2     10.0.1.2     980  0x80000004 0x5ec5
10.0.2.1     10.0.1.2     1750 0x80000003 0x4dd8
10.0.3.2     10.0.3.2     1733 0x80000003 0x67b5
10.0.4.2     10.0.5.1     1313 0x80000004 0x5eba

...skipping one line
10.0.6.2     10.0.6.2     1262 0x80000004 0x45cf
10.0.7.1     10.0.6.2     1732 0x80000003 0x55b6
10.0.8.2     10.0.8.2     1311 0x80000004 0x4cc0
10.0.9.1     10.0.8.2     901  0x80000004 0x27e6
10.0.10.2    10.0.8.2     581  0x80000004 0x5bab
```

Figure 14: show ip ospf database on n2 — Before any changes

```
n2# show ip ospf route
===== OSPF network routing table =====
N   10.0.0.0/24      [20] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.2.2, eth1
N   10.0.1.0/24      [10] area: 0.0.0.0
    directly attached to eth0
N   10.0.2.0/24      [10] area: 0.0.0.0
    directly attached to eth1
N   10.0.3.0/24      [30] area: 0.0.0.0
    via 10.0.9.1, eth2
N   10.0.4.0/24      [30] area: 0.0.0.0
    via 10.0.9.1, eth2
N   10.0.5.0/24      [40] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.6.0/24      [20] area: 0.0.0.0
    via 10.0.1.1, eth0
N   10.0.7.0/24      [30] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.8.0/24      [20] area: 0.0.0.0
    via 10.0.9.1, eth2
N   10.0.9.0/24      [10] area: 0.0.0.0
    directly attached to eth2
N   10.0.10.0/24     [20] area: 0.0.0.0
    via 10.0.9.1, eth2
N   10.0.11.0/24     [20] area: 0.0.0.0
    via 10.0.2.2, eth1
N   10.0.12.0/24     [40] area: 0.0.0.0
    via 10.0.9.1, eth2
N   10.0.13.0/24     [20] area: 0.0.0.0
    via 10.0.9.1, eth2

===== OSPF router routing table =====
===== OSPF external routing table =====

n2#
```

Figure 15: show ip ospf route on n2 — Before any changes

- The routing table shows all subnets reachable through various next-hops via n4 (10.0.9.1) and other routers, with costs ranging from [10] to [50].

Question 6.iii: OSPF Updates After Cost Change on n4

- We changed the OSPF cost on n4 eth1 to 20. Immediately, Wireshark captured a flood of LS Update (LSU) packets propagating across all routers, as shown in (Figure 16).

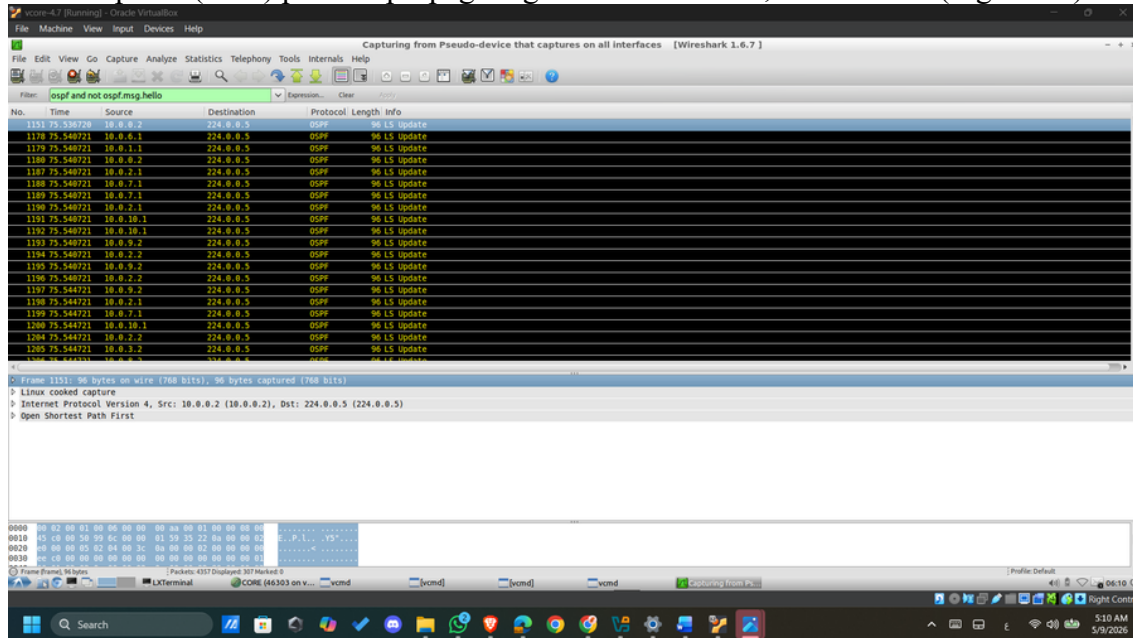


Figure 16: Wireshark — LS Update Packets Flooding the Network After Cost Change on n4

- **Comment:** The LSU packets are OSPF's mechanism for propagating Link State Advertisements (LSAs) across the entire OSPF area. Each router that receives an LSU recomputes its routing table using Dijkstra's algorithm and floods the update to its neighbors, until the entire network converges to a new consistent state.

Question 6.iv: OSPF Updates After Bringing Down n4 Interfaces

- We brought down all interfaces on n4 (eth0, eth2). This caused n4 to disappear from the OSPF topology. Wireshark captured a second wave of LSU packets (larger, 264 bytes) along with LS Acknowledge packets, as shown in (Figure 17).

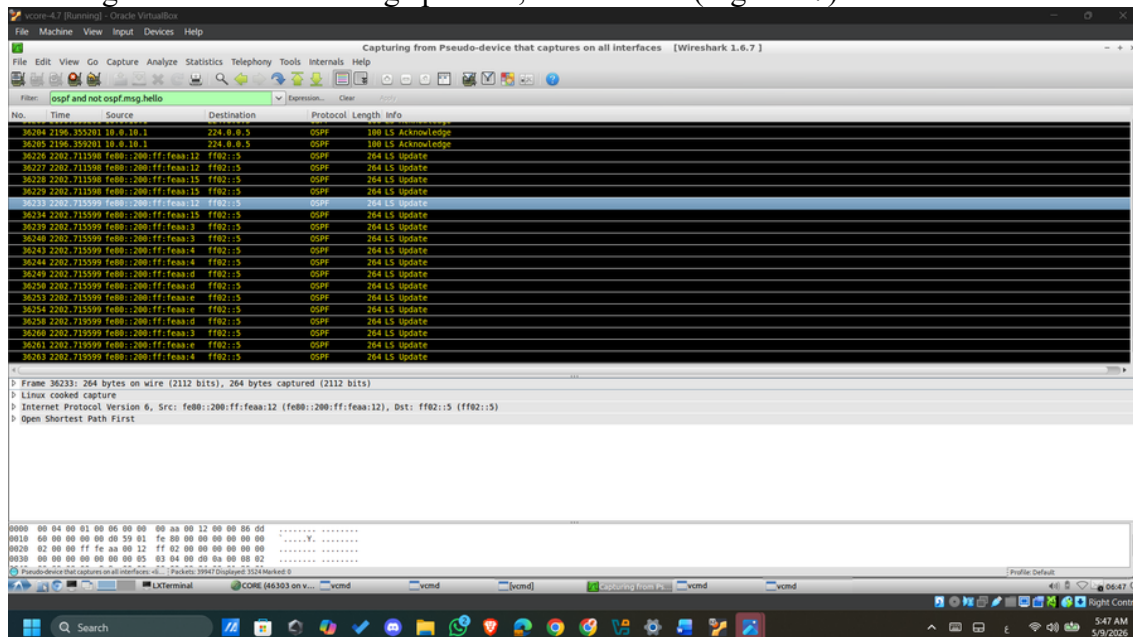


Figure 17: Wireshark — LS Update (264 bytes) and LS Acknowledge After n4 Interfaces Down

- **Comment:** The larger LSU packets (264 bytes vs 96 bytes before) indicate that each LSA now carries topology change information for multiple links. The LS Acknowledge packets confirm that neighboring routers received and accepted the updates. OSPF detected the failure through the Dead timer expiry and triggered a full SPF recalculation.

Question 6.v: OSPF Database After n4 Goes Down

- After n4 went down, we re-ran show ip ospf database and show ip ospf route on n2 to observe the updated state, as shown in (Figures 18 and 19).

The image contains two terminal screenshots. The left screenshot shows the output of the 'show ip ospf database' command on router n2. It displays the OSPF Router with ID (10.0.1.2) and lists Link States and Net Link States for Area 0.0.0.0. The right screenshot shows the output of the 'show ip ospf route' command on router n2, displaying the OSPF network routing table with various routes and their associated costs and next hops.

```

n2# show ip ospf database
Hello, this is Quagga (version 0.99.20.1).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

n2# show ip ospf database

    OSPF Router with ID (10.0.1.2)

        Router Link States (Area 0.0.0.0)

Link ID      ADV Router   Age  Seq#       CkSum  Link count
10.0.0.1     10.0.0.1     217  0x8000000e 0xeac1  3
10.0.0.2     10.0.0.2     117  0x8000000e 0x920c  3
10.0.1.2     10.0.1.2     146  0x8000000e 0x850e  3
10.0.3.1     10.0.3.1     881  0x80000015 0x52a7  1
10.0.3.2     10.0.3.2     869  0x8000000f 0xecaa  3
10.0.5.1     10.0.5.1     80   0x8000000d 0x8a0a  3
10.0.6.2     10.0.6.2     198  0x8000000d 0xc4af  3
10.0.8.2     10.0.8.2     847  0x8000000f 0x2425  4

        Net Link States (Area 0.0.0.0)

Link ID      ADV Router   Age  Seq#       CkSum
10.0.0.2     10.0.0.2     1197 0x80000004 0x59ce
10.0.1.2     10.0.1.2     396  0x80000005 0x5cc6
10.0.2.1     10.0.1.2     1156 0x80000004 0x4bd9
10.0.4.2     10.0.5.1     720  0x80000005 0x5cbb
10.0.5.1     10.0.5.1     1109 0x80000004 0x63b4
...skipping one line
10.0.7.1     10.0.6.2     1108 0x80000004 0x53be
10.0.9.1     10.0.8.2     297  0x80000005 0x25e7
10.0.10.2    10.0.8.2     1777 0x80000004 0x5bab

n2# show ip ospf route
===== OSPF network routing table =====
N    10.0.0.0/24      [20] area: 0.0.0.0
      via 10.0.1.1, eth0
N    10.0.1.0/24      [10] area: 0.0.0.0
      via 10.0.2.2, eth1
      [10] area: 0.0.0.0
      directly attached to eth0
N    10.0.2.0/24      [10] area: 0.0.0.0
      directly attached to eth1
N    10.0.3.0/24      [40] area: 0.0.0.0
      via 10.0.1.1, eth0
      via 10.0.9.1, eth2
N    10.0.4.0/24      [50] area: 0.0.0.0
      via 10.0.1.1, eth0
      via 10.0.9.1, eth2
N    10.0.5.0/24      [40] area: 0.0.0.0
      via 10.0.1.1, eth0
      via 10.0.9.1, eth2
N    10.0.6.0/24      [20] area: 0.0.0.0
      via 10.0.1.1, eth0
N    10.0.7.0/24      [30] area: 0.0.0.0
      via 10.0.1.1, eth0
      via 10.0.9.1, eth2
N    10.0.8.0/24      [20] area: 0.0.0.0
      via 10.0.9.1, eth2
...skipping one line
N    10.0.10.0/24     directly attached to eth2
      [20] area: 0.0.0.0
      via 10.0.9.1, eth2
N    10.0.11.0/24     [20] area: 0.0.0.0
      via 10.0.2.2, eth1
N    10.0.12.0/24     [50] area: 0.0.0.0
      via 10.0.1.1, eth0
      via 10.0.9.1, eth2
N    10.0.13.0/24     [20] area: 0.0.0.0
      via 10.0.9.1, eth2
===== OSPF router routing table =====
===== OSPF external routing table =====
  
```

Figure 18: show ip ospf database on n2 — After n4 goes down

Figure 19: show ip ospf route on n2 — After n4 goes down

- Comparing the routing tables before and after, the routes that previously passed through n4 (10.0.9.1) have been updated to use alternative paths, and some routes now show higher costs. The OSPF database no longer contains LSAs advertised by n4, confirming successful convergence after the topology change.
- **Comment:** OSPF's link-state database ensures that every router has a consistent view of the network topology. When n4 failed, the LSA flooding mechanism rapidly propagated the topology change, and Dijkstra's algorithm computed new optimal paths automatically. This demonstrates OSPF's key advantage: fast and automatic convergence in response to topology changes, without any manual intervention.